

MINECRAFT

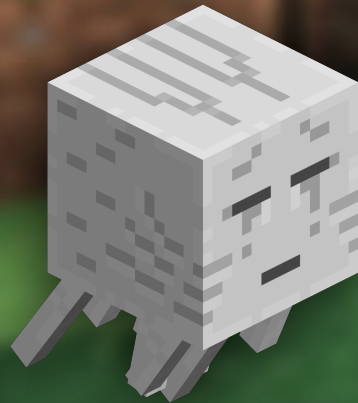
DATABASE PROJECT

ISE 305

Options

Quit

Anna Li, William Lee,
Prarthana Biswas



MINECRAFT



Loading.....

INTRODUCTION

Minecraft is a sandbox video game developed by Mojang Studios. The game was created by Markus "Notch" Persson in the Java programming language. We are creating a database for Minecraft, focusing on organizing and connecting in-game data. Minecraft has various types of things like items, blocks, food, mobs, recipes, enchantments, and more. Each of these will have its own table in the database, and we'll set up relationships between them to represent how they interact in the game. For instance, items are used in recipes to create new items, mobs can drop certain items, and items can be classified as food or blocks. By structuring it this way, our database will clearly show the connections between these elements, making it easier to analyze and manage in-game data.

PROJECT GOALS

Our project has several goals:

- Collect and organize all the data required for designing the database effectively
- Set up tables for storing and managing the data
- Define relationships between entities in the database
- Establish "objects" as the central entity in the database design
- Connect "objects" to related entities, including:
 - Recipes
 - Blocks
 - Food





TEAM CONTRIBUTION



ANNA LI

- Project Statement
- RDM
- Research and formatting Data
- Writing and generating Query Questions
- Slides development

WILLIAM LEE

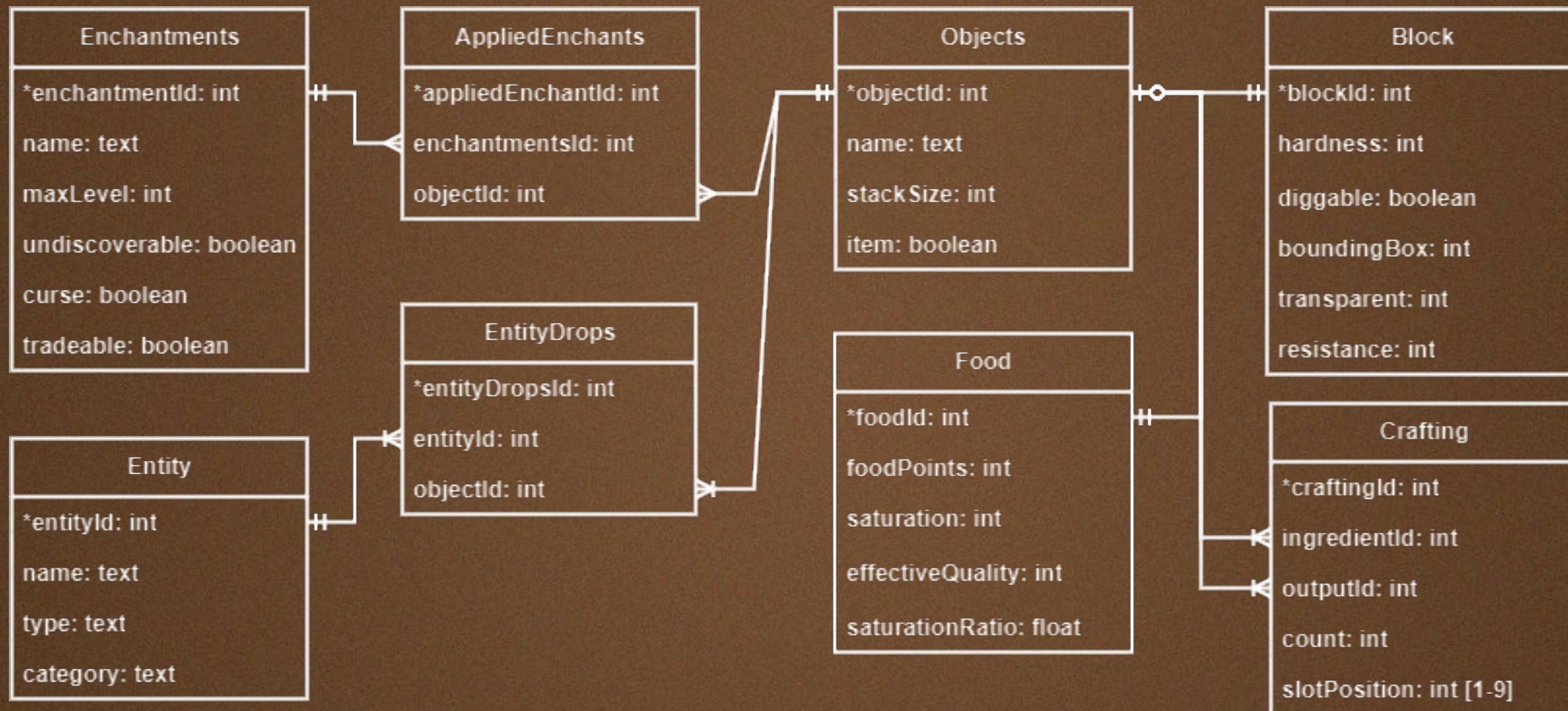
- Project Statement
- RDM
- Research and formatting Data
- Writing and generating Query Questions
- Slides development

PRARTHANA BISWAS

- Project Statement
- RDM
- Research and formatting Data
- Writing MS access reports and Forms
- Slides development



RDM



Multiplicity Examples

One-to-One

Objects to Blocks

Relationship: One **object** (**objectId**) can represent one **block** (**blockId**).

Multiplicity:

Each record in **Objects** is linked to at most one record in **Blocks**.

Each record in **Blocks** corresponds to exactly one record in **Objects**.

One-to-Many

Enchantments to AppliedEnchants

Relationship: One **enchantment** (**enchantmentId**) can be applied to many **objects** through **AppliedEnchants**.

Multiplicity:

Each record in **Enchantments** can relate to zero, one, or many records in **AppliedEnchants**.

Each record in **AppliedEnchants** corresponds to exactly one record in **Enchantments**.

One-to-Many/Many-to-Many

Crafting to Objects

Relationship: Each crafting recipe (**craftingId**) uses multiple objects (**ingredientId**).

Multiplicity:

Each record in **Crafting** can relate to one or many objects as **ingredients**.

Each **object** (**objectId**) can be used in many **crafting recipes**, making this a many-to-many relationship (with **Crafting** serving as the junction table).



Question 1

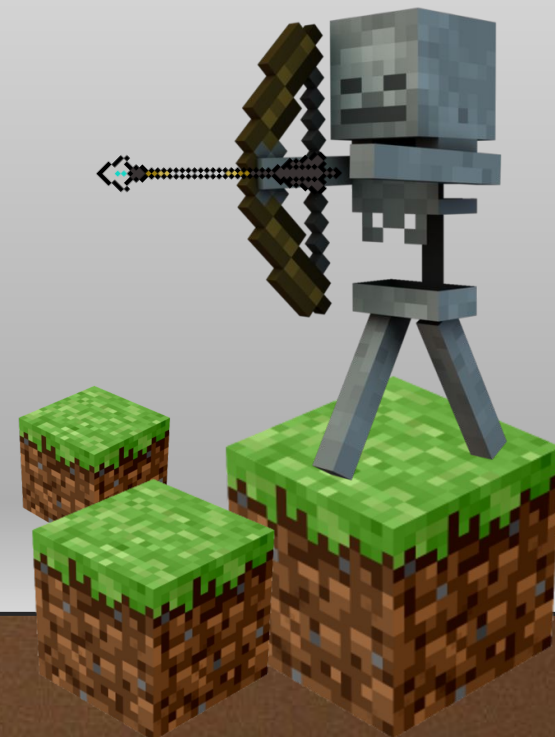
Treasure

Fight

Enchant

Food

How many enchants that are for diamond boots can only be found outside of the enchantment table?



Answer 1

```
SELECT COUNT(*) AS numOfEnchants
FROM (Objects INNER JOIN AppliedEnchants ON
Objects.objectId = AppliedEnchants.objectId) INNER JOIN
Enchantments ON Enchantments.enchantmentId =
AppliedEnchants.enchantmentId
WHERE Objects.name = 'diamond_boots' AND
Enchantments.undiscoverable = True;
```

numOfEnchants ▼
4

Question 2

Treasure

Fight

Enchant

Food

What Minecraft entities would be necessary to fight to obtain the ingredients needed to craft leads?



ANSWER 2

```
SELECT Entities.name AS entityName, COUNT(Entities.name) AS numNeeded,  
Objects.name AS ingredient  
FROM (Entities INNER JOIN EntityDrops ON Entities.entityId =  
EntityDrops.entityId) INNER JOIN (Objects INNER JOIN Crafting ON  
Objects.objectId = Crafting.ingredientId) ON Objects.objectId =  
EntityDrops.objectId  
WHERE Crafting.outputId IN (  
    SELECT objectId  
    FROM Objects  
    WHERE name = "lead"  
)  
GROUP BY Entities.name, Objects.name;
```

entityName ▾	numNeeded ▾	ingredient ▾
slime	1	slime_ball
spider	4	string

Question 3

Treasure

Fight

Enchant

Food

What are all the enchants that I can
put on my diamond sword?



Answer 3

```
SELECT Enchantments.name
FROM (Objects INNER JOIN AppliedEnchants ON Objects.id =
AppliedEnchants.objectId) INNER JOIN Enchantments ON
Enchantments.enchantmentId =
AppliedEnchants.enchantmentId
WHERE Objects.name = 'diamond_sword';
```

name
sharpness
smite
bane_of_arthropods
knockback
fire_aspect
looting
sweeping
unbreaking
mending
vanishing_curse

Question 4

Treasure

Fight

Enchant

Food

I want to find the food that will fill me the most and the longest. This is called effective quality where is Food Points + Saturation. And I like to have the ingredients to make it, if it is craftable



Answer 4

```
SELECT
    Foods.foodId AS foodId,
    FoodObjects.displayName AS foodDisplayName,
    Ingredients.ingredientId AS ingredientsId,
    IngredientObjects.displayName AS ingredientsDisplayName,
    COUNT(Ingredients.ingredientId) AS count
FROM Foods, Objects AS FoodObjects, Crafting AS Ingredients, Objects AS IngredientObjects
WHERE
    Foods.foodId = FoodObjects.objectId
    AND FoodObjects.objectId = Ingredients.outputId
    AND Ingredients.ingredientId = IngredientObjects.objectId
    AND Foods.effectiveQuality = (SELECT MAX(effectiveQuality) FROM Foods)
GROUP BY Foods.foodId, FoodObjects.displayName, Ingredients.ingredientId,
IngredientObjects.displayName;
```

	foodId ▾	foodDisplayName ▾	ingredientsId ▾	ingredientsDisplayName ▾	count ▾
	413	Rabbit Stew	39	Brown Mushroom	1
	413	Rabbit Stew	281	Bowl	1
	413	Rabbit Stew	391	Carrot	1
	413	Rabbit Stew	393	Baked Potato	1
	413	Rabbit Stew	412	Cooked Rabbit	1

Question 5

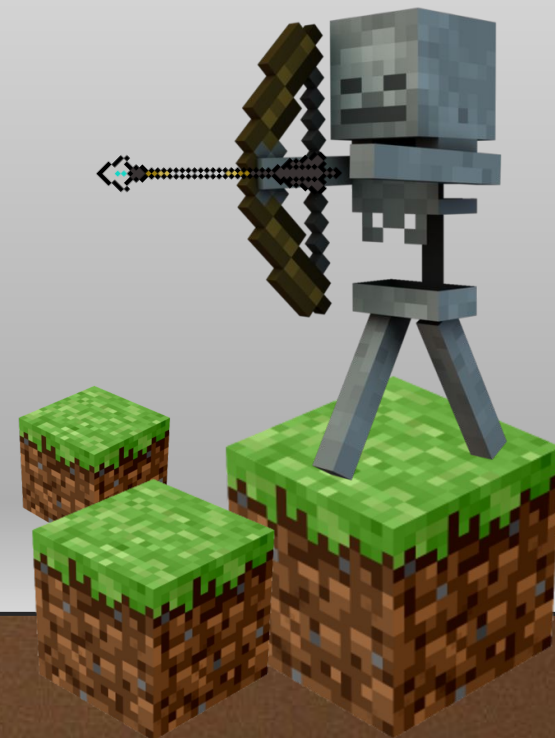
Blocks

Hey

There

Pa1

Give me blocks that are
craftable sorted by highest
blast resistance



Answer 5

```
SELECT Blocks.blockId, Objects.displayName,  
       Blocks.resistance  
FROM Blocks, Objects, Crafting  
WHERE  
       Blocks.blockId = Objects.objectId  
       AND Objects.objectId = Crafting.outputId  
GROUP BY Blocks.blockId, Objects.displayName,  
         Blocks.resistance  
ORDER BY Blocks.resistance DESC;
```

blockId	displayName	resistance
116	Enchantment Ta	1200
145	Anvil	1200
130	Ender Chest	600
41	Block of Gold	6
42	Block of Iron	6
44	Stone Slab	6
45	Brick	6
57	Block of Diamon	6
67	Cobblestone Sta	6
84	Lukebox	6

FORMS 1 - Blocks

Displays Block details such as
blockId, hardness, diggable,
boundingBox, transparent,
and resistance.

Objects1

objectId	1
displayName	Stone
name	stone
stackSize	64
item	True

Blocks

Blocks

blockId	1
hardness	1.5
diggable	True
boundingBox	block
transparent	False
resistance	6

Records: 1 of 1

No Filter

Search

FORMS 2 – Food

Displays Food details such as foodId, foodPoints, saturation, and effectiveQuality

Objects2

objectId	103
displayName	Melon
name	melon_block
stackSize	64
item	True

Food

foodId	103
foodPoints	2
saturation	1.2
effectiveQuality	3.2

REPORT

This report compares food items based on their quality and saturation levels; here you can see it is descending by foodPoints and effectiveQuality.

foodanalysis

displayName	foodPoints	saturation	effectiveQuality	saturationRatio
Rabbit Stew	10	12	22	1.2
Cooked Porkchop	8	12.8	20.8	1.6
Steak	8	12.8	20.8	1.6
Golden Carrot	6	14.4	20.4	2.4
Cooked Mutton	6	9.6	15.6	1.6
Golden Apple	4	9.6	13.6	2.4

Constraints and Validation

General Constraints

- Primary Keys: Ensure each table has a unique identifier (e.g., **objectId** in **Objects**).
- Foreign Keys: Enforce relationships between tables (e.g., **blockId** in **Blocks** references **objectId** in **Objects**).
- Not Null: Essential fields cannot be empty (e.g., **name** in **Objects**).
- Data Types: Each field has an appropriate data type (e.g., **integers**, **booleans**, **text**).

Validation Rules

- Numerical Fields:
 - **hardness** (**Blocks**): Must be ≥ 0 .
 - **stackSize** (**Objects**): Must be between 1 and 64.
 - **foodPoints** (**Food**): Must be ≥ 0 .
 - **slotPosition** (**Crafting**): Must be between 1 and 9.
- Boolean Fields:
 - **diggable** (**Blocks**) and item (**Objects**): Only True or False allowed.
- Relationships:
 - Foreign keys (e.g., **foodId** in **Food**) must match valid primary keys in related tables.

Summary

These constraints and validation rules ensure:

- Data consistency and integrity.
- Proper relationships between tables.
- Prevention of invalid data entries.

Additional Features

→ **Transparency for Blocks**

- ◆ Our database includes a transparency attribute for blocks, allowing differentiation between opaque blocks like stone and transparent ones like glass. This improves light interaction and adds realism to gameplay.

→ **Bounding Blocks**

- ◆ We define bounding properties for blocks, enabling more accurate collision detection and interaction. This feature is especially useful for irregular blocks like stairs and slabs.

→ **Curses**

- ◆ Curses add a layer of challenge by introducing negative effects, such as reduced durability or item trapping. Players must weigh the benefits of powerful items against the risks of curses.

→ **Max Level for Enchantment Blocks**

- ◆ A max enchantment level attribute ensures balanced gameplay by limiting enchantments for specific blocks. Players can unlock higher levels for rare materials, encouraging progression.

CONCLUSION + Q & A

In conclusion, our Minecraft Database Project provides an innovative and efficient way to organize and manage in-game data, enhancing the user experience and gameplay. We appreciate your attention and are excited about the possibilities this project brings to the Minecraft community. Thank you for listening, and we would be happy to answer any questions you may have.



THANK
YOU

Quit

Restart



THANK

Are you sure you
want to quit ?

Yes

No

Restart

SOURCES

- PrismarineJS. (n.d.). Minecraft-Data/data/PC/1.12/blocks.json at master · prismarinejs/minecraft-data. GitHub.
<https://github.com/PrismarineJS/minecraft-data/tree/master/data/pc/1.12>
- Free interactive minecraft template powerpoint and Google Slides. SlideChef. (2024, October 5).
<https://slidechef.net/templates/free-interactive-minecraft-template/>

